

Engineering Guide & Sprint Plan

PRODUCT NAME	Staye
DOCUMENT TITLE	Backend Infrastructure: Supabase to AWS Migration Runbook
REVISION	Revision 2 (Detailed UI Walkthrough & Staging Topology)
TARGET ARCHITECTURE	Phase 1 (EC2, RDS PostgreSQL, Cognito, S3)
TARGET AUDIENCE	Backend Engineering & DevOps Team
PREPARED BY	Zenxvio Engineering Team
DATE	July 2026

Purpose of This Document

This runbook provides exhaustive, step-by-step UI instructions for the engineering team to migrate the Staye platform's backend infrastructure off Supabase and onto a production-grade AWS environment.

The migration is structured into **four distinct Sprints**. By the end of Sprint 4, the application will run as a Dockerized Next.js monolith on a free-tier EC2 instance, utilizing AWS RDS for data, Cognito for authentication, and S3 for storage.

Table of Contents

- 1. Migration Architecture Overview
- 2. Sprint 1: Cloud Foundation & Infrastructure Provisioning
- 3. Sprint 2: Codebase Refactoring (Moving off Supabase)
- 4. Sprint 3: Dockerization & CI/CD Pipeline
- 5. Sprint 4: Data Cutover & Go-Live

1. Migration Architecture Overview

1.1 What Stays & What Changes

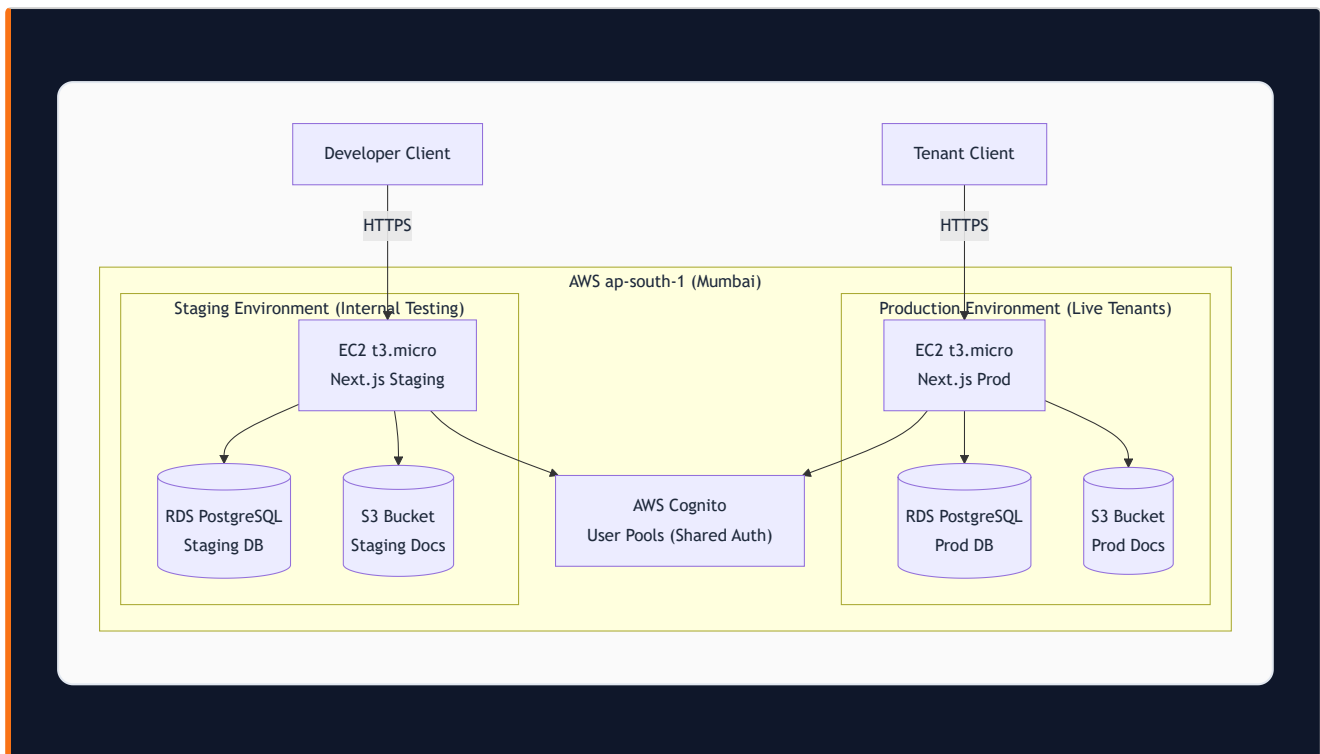
Because Staye is built on **Next.js (App Router)** and utilizes the **Prisma ORM**, the core business logic remains entirely intact.

- **What DOES NOT change:** Database schemas, Prisma queries (`prisma.tenant.findUnique()`), API route business logic, and UI components.
- **What MUST be rewritten:**
 1. Authentication logic (migrating from `supabase.auth` to AWS Cognito).
 2. File upload logic (migrating from `supabase.storage` to `@aws-sdk/client-s3`).
 3. Environment variables and CI/CD pipelines.

1.2 Target Topology: Dual Environment (Staging & Prod)

To ensure developers never test code on live tenant data, we are separating Staging and Production.

⚠️ **CRITICAL BILLING WARNING** ⚠️ The AWS Free Tier covers exactly *one* EC2 server and *one* RDS database. If you spin up both the Staging and Production servers in AWS simultaneously, you will exceed the free tier limit and incur charges (approx. \$15-\$25/month). If you require \$0.00 costs, developers must run the Staging database locally on their machines using Docker Compose instead of AWS RDS.



2. Sprint 1: Cloud Foundation & Infrastructure Provisioning

Objective: Securely configure the AWS environment and provision empty resources. No code changes happen in this sprint.

Step 2.0: Budget & Billing Alarms (CRITICAL)

Before doing anything, you must set up an AWS Budget to prevent surprise bills if the free tier is breached.

1. Log in to AWS as the **Root User**.
2. Search for **AWS Budgets** in the top search bar.
3. Click **Create budget** → **Use a template (simplified)** → **Zero spend budget**.
4. Set the email address to the founder's email.
5. Click **Create budget**. You will now receive an email the second your AWS bill hits \$0.01.

Step 2.1: Root Security & IAM Configuration

1. **Secure the Root Account:** Enable MFA (Multi-Factor Authentication) on the Root account immediately.
2. **Create the Developer IAM Policy:**
 - Search for **IAM** → Click **Policies** → **Create policy**.
 - Click the **JSON** tab (Do NOT use the visual editor, or you will get "resources not specified" errors).
 - Paste this exact JSON payload (this strictly limits the developer while allowing them to build the MVP, bypassing `kms` and `iam:PassRole` security warnings):

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "ec2:*",
        "rds:*",
        "s3:*",
        "cognito-idp:*",
        "ecr:*",
        "iam:CreateServiceLinkedRole"
      ],
      "Resource": "*"
    },
    {
      "Effect": "Allow",
      "Action": "iam:PassRole",
      "Resource": "*",
      "Condition": {
        "StringEquals": {
          "iam:PassedToService": [
            "rds.amazonaws.com",
            "ec2.amazonaws.com"
          ]
        }
      }
    },
    {
      "Effect": "Deny",
      "Action": [
        "aws-portal:*",
        "iam:CreateUser",
        "iam:DeleteUser"
      ],
      "Resource": "*"
    }
  ]
}

```

- Name the policy `Staye-Developer-Policy` and click **Create policy**.

3. Create IAM User:

- Go to IAM → Users → **Create user**.
- Name it `staye-developer` and grant AWS Management Console access.
- Select **Attach policies directly** and attach the `Staye-Developer-Policy`.

- After creation, go to the user's **Security credentials** tab, generate an **Access Key (CLI)**, and save the Secret Key securely in your `.env` file.
 - **Log out of Root, and log back in as `stave-developer` for all subsequent steps.**
-

Step 2.2: VPC & Network Security

Security groups act as virtual firewalls. *Note: AWS reserves the `sg-` prefix, so we use `staye-` prefixes.*

1. Create the Web Firewall (`staye-web-sg`):

- Search for **EC2** → Scroll down the left menu to **Security Groups** → **Create security group**.
- Name: `staye-web-sg`. VPC: Leave as default.
- Inbound Rules:
 - HTTP (Port 80) from `0.0.0.0/0`
 - HTTPS (Port 443) from `0.0.0.0/0`
 - SSH (Port 22) from `My IP` (Only the developer can access the server terminal).
- Click **Create security group**.

2. Create the Database Firewall (`staye-db-sg`):

- Click **Create security group** again.
 - Name: `staye-db-sg`.
 - Inbound Rules:
 - PostgreSQL (Port 5432) → In the source search box, explicitly type and select `staye-web-sg`.
 - *This ensures only your EC2 web server can talk to the database. The public internet is completely blocked.*
 - Click **Create security group**.
-

Step 2.3: Resource Provisioning (Exhaustive UI Guide)

1. Amazon RDS (The Database)

If you misconfigure this section, you will be billed hundreds of dollars. Follow these UI steps exactly to stay on the Free Tier.

1. Search for **RDS** → Click **Create database**.
2. Click the dropdown next to the button and ensure **Full configuration** is selected.
3. Choose **Standard create** and **PostgreSQL**.
4. Leave Templates on **Production**, but override the following settings manually:
5. **Availability and durability**: Explicitly select **Single-AZ DB instance deployment (1 instance)** on the far right.
6. **Credentials Settings**: Change from "Managed in AWS Secrets Manager" (which costs money) to **Self managed**. Type your own secure master password.
7. **Instance configuration**: Change the dropdown to **Burstable classes (includes t classes)**. Select `db.t3.micro` or `db.t4g.micro`.
8. **Storage**: Change the dropdown from "Provisioned IOPS SSD (io2)" to **General Purpose SSD (gp2)**. Change the size from 400 down to **20 GiB**. Uncheck "Enable storage autoscaling".
9. **Connectivity**:
 - Public access: **No**.
 - VPC security group: Click the X to remove `default`, and select `staye-db-sg`.
10. **Additional Configuration (Very Bottom)**:
 - Initial database name: Type `staye_db`.
 - Backup**: Uncheck "Enable automated backup" (Because 20GB of data + 7 days of backups exceeds the 20GB free tier limit).
 - Encryption**: Uncheck "Enable encryption" (This prevents the `kms:ListAliases` error).
 - Monitoring**: Uncheck "Enable Performance Insights" and "Enable Enhanced monitoring" (These cost money and will throw IAM errors if left checked).
11. Scroll down, verify the Estimated Monthly Costs mentions the "Free tier", and click **Create database**.

2. Amazon S3 (Storage Bucket)

1. Search for **S3** → Click **Create bucket**.
2. Bucket name: `staye-production-documents-xyz` (Must be globally unique). Region: `ap-south-1`.
3. Ensure **Block all public access** is **CHECKED**.
4. Click **Create bucket**.

3. Amazon Cognito (Authentication)

1. Search for **Cognito** → Click **Create user pool**.

2. **Step 1:** Check **Email** for sign-in.
3. **Step 2:** Leave Password defaults. Select **No MFA** for the MVP.
4. **Step 3:** **UNCHECK "Enable self-registration"**. (Tenants are added by wardens/booking flow only).
5. **Step 4:** Select **Send email with Cognito** (Free).
6. **Step 5:** User pool name: `staye-user-pool`. Initial app client: **Confidential client** named `staye-nextjs-client`.
7. Click **Create user pool**.

4. Amazon ECR (Container Registry)

1. Search for **ECR** (Elastic Container Registry) → Click **Create repository**.
 2. Visibility: **Private**. Name: `staye-backend`.
 3. Check **Scan on push** for free security scanning.
 4. Click **Create repository**.
-

3. Sprint 2: Codebase Refactoring (Moving off Supabase)

Objective: Rip out Supabase SDK dependencies and wire the Next.js application to the newly provisioned AWS resources.

Step 3.1: Database Connection Update

1. Obtain the Endpoint URL from the newly created AWS RDS instance.
2. Update the `.env` file:

```
# Old: DATABASE_URL="postgresql://postgres:password@db.supabase.co:5432/postgres"
# New:
DATABASE_URL="postgresql://postgres:YOUR_PASSWORD@your-rds-endpoint.ap-south-1.rds.amazonaws.com:5432/staye_db"
```

3. Run `npm run prisma db push` (or `migrate deploy`) against the new AWS RDS database to generate the tables.

Step 3.2: Authentication Migration (Cognito)

1. **Remove Supabase Auth:** Uninstall `@supabase/supabase-js` and `@supabase/ssr`.
2. **Install AWS Auth:** Install `aws-amplify` (preferred for Cognito frontend logic) or configure `next-auth` (Auth.js) with the Cognito provider.
3. **Refactor Middleware:**
 - Open `middleware.ts`.
 - Replace the Supabase session check with a Cognito JWT verification logic. If no valid Cognito token is found, redirect to `/login`.
4. **Refactor Login Page:** Update the UI to pass email/password to Cognito instead of Supabase.

Step 3.3: Storage Migration (S3)

1. **Install SDK:** `npm install @aws-sdk/client-s3 @aws-sdk/s3-request-presigner`
2. **Refactor Upload Logic:**
 - Locate all API routes handling KYC document and Stay Pass uploads.
 - Replace Supabase `upload()` with AWS SDK `PutObjectCommand`.
3. **Refactor Fetch/View Logic:**

- Locate where PDFs and images are rendered.
 - Replace Supabase public URL logic with AWS SDK `getSignedUrl(new GetObjectCommand( ... ))` with a 1-hour expiration.
-

4. Sprint 3: Dockerization & CI/CD Pipeline

Objective: Containerize the application and automate the deployment to EC2 so developers never have to manually transfer files.

Step 4.1: The Dockerfile

Create a production-ready `Dockerfile` in the root of the project utilizing multi-stage builds to keep the image size minimal.

```
# Base node image
FROM node:18-alpine AS base
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci

# Build stage
FROM base AS builder
COPY . .
# Generate prisma client for linux
RUN npx prisma generate
RUN npm run build

# Production stage
FROM node:18-alpine AS runner
WORKDIR /app
COPY --from=builder /app/package.json ./
COPY --from=builder /app/.next/standalone ./
COPY --from=builder /app/.next/static ./next/static
COPY --from=builder /app/public ./public

EXPOSE 3000
CMD ["node", "server.js"]
```

Note: Ensure `output: 'standalone'` is set in `next.config.js`.

Step 4.2: GitHub Actions CI/CD

Create `.github/workflows/deploy.yml`. The pipeline must execute the following steps automatically when code is pushed to the `main` branch:

1. **Build:** Run `docker build`.

2. **Authenticate:** Use `aws-actions/configure-aws-credentials` to log into AWS.
 3. **Push:** Push the built Docker image to the AWS ECR repository.
 4. **Deploy:** Use AWS Systems Manager (SSM) Run Command to trigger a script on the EC2 instance that:
 - Pulls the latest image from ECR.
 - Stops the old container.
 - Starts the new container injecting secrets from SSM Parameter Store (This avoids paying for AWS Secrets Manager).
-

5. Sprint 4: Data Cutover & Go-Live

Objective: Safely migrate existing production data from Supabase to AWS with minimal downtime, and flip the DNS switch to go live.

Step 5.1: The Maintenance Freeze

1. Schedule a 2-hour maintenance window during off-peak hours (e.g., 2:00 AM IST).
2. Deploy a maintenance banner to the frontend to prevent users from modifying data.
3. Revoke application access to the Supabase database to guarantee no new writes occur during the copy process.

Step 5.2: Database Migration

1. **Export from Supabase:** Use `pg_dump` to extract the entire database schema and data.

```
pg_dump -U postgres -h db.supabase.co -F c -d postgres > stayer_backup.dump
```

2. **Import to AWS RDS:** Use `pg_restore` to push the data into the new RDS instance.

```
pg_restore -U postgres -h your-rds-endpoint.amazonaws.com -d stayer_db < stayer_backup.dump
```

Step 5.3: Storage Migration

1. Write a temporary Node.js script using the Supabase SDK to list and download all existing KYC documents and PDFs to a local machine.
2. Extend the script to upload those downloaded files directly into the new AWS S3 `stayer-production-documents` bucket, maintaining the exact same folder structure and filenames.

Step 5.4: Auth User Cutover

Warning: Passwords cannot be exported from Supabase due to hashing security.

1. Export the list of user emails and roles from Supabase Auth.
2. Write a script to create these users in AWS Cognito using `adminCreateUser`.
3. Set their accounts to require a password reset on first login.

- 4. Tenant Communication:** Send an automated email/SMS to all existing users: *"Staye has upgraded our systems. Please click here to set a new password for your account."*

Step 5.5: DNS Flip (Go-Live)

1. Log into your domain registrar (or Route 53 if managing DNS there).
 2. Update the `A` record (or `CNAME`) for `app.staye.com` to point to the Elastic IP of the AWS EC2 instance.
 3. Once DNS propagates, verify the platform is running on the AWS infrastructure.
 4. Disable the old Supabase project to prevent accidental usage and stop billing.
-

End of Runbook